

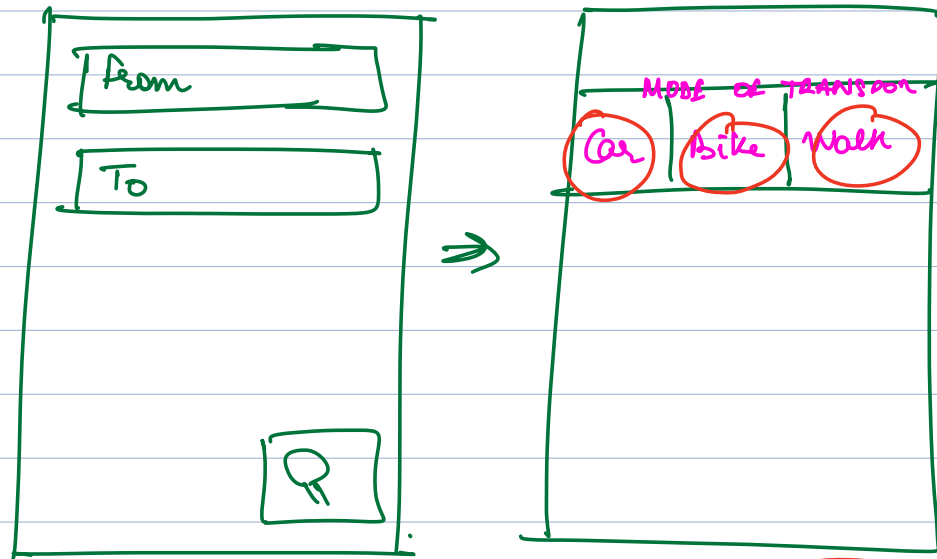
Observer and Strategy Design Patterns

Behavioural Design Patterns

→ Solve common problems wrt intricacies of behaviours/ functionalities to be supported by our software system.

STRATEGY DESIGN PATTERN

Google Maps



→ Shortest by time path

→ Shortest by dist path

→ path with no toll

→ path with less traffic

find Path $\Rightarrow$ functionality

supported by google maps

find Path (to, from)

$\rightarrow$ functionality can be implemented in multiple ways.

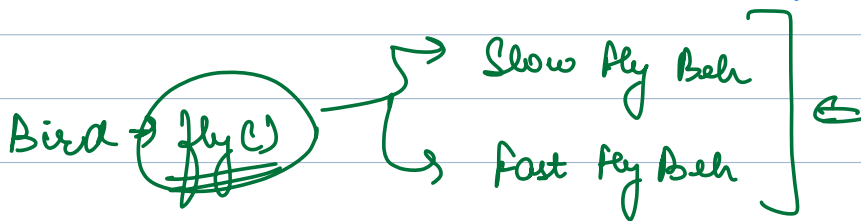
When?

There is a functionality that can be implemented in diff ways.

whenever you see "diff ways to do this"

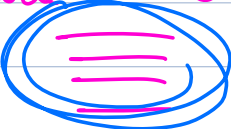
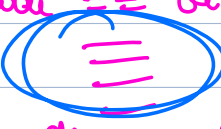


Strategy Design Pattern



$\hookrightarrow$ Strategy design pattern

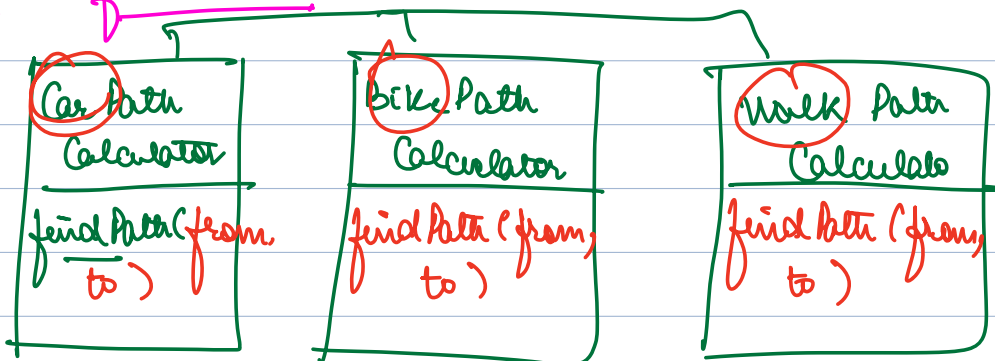
class Google Maps {

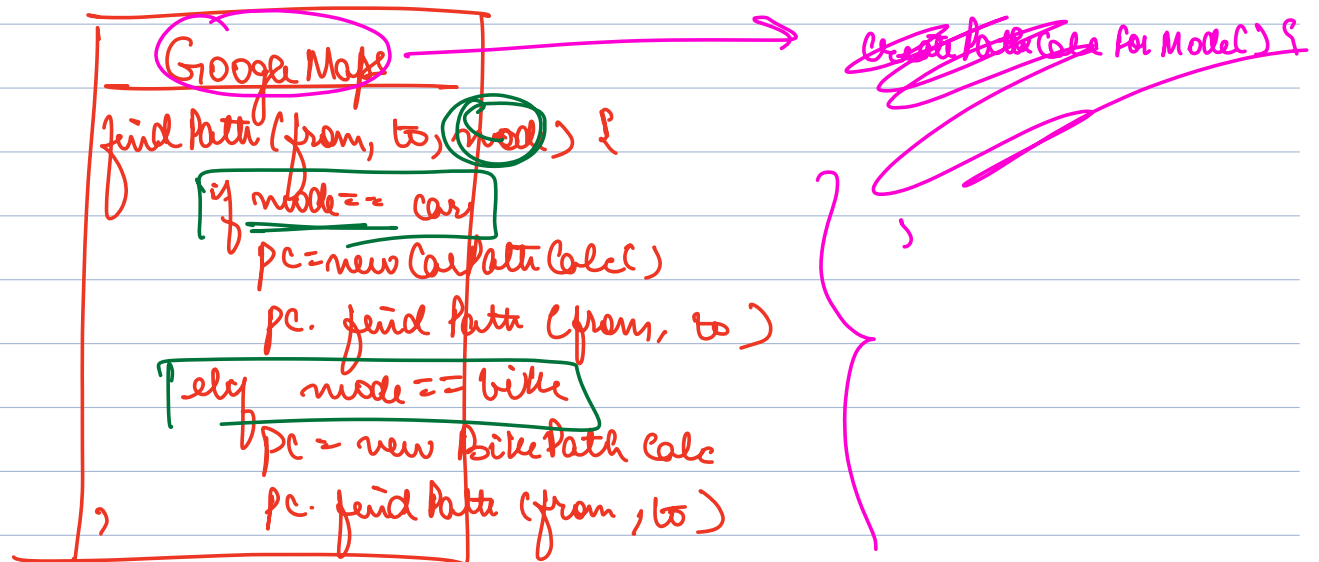
```
    findPath (from, to, mode) {  
        => if (mode == car)  
            =>   
        => elif (mode == bike)  
            =>   
        => elif (mode == walk) {  
            =>   
        }  
    }  
}
```

Or Switch case

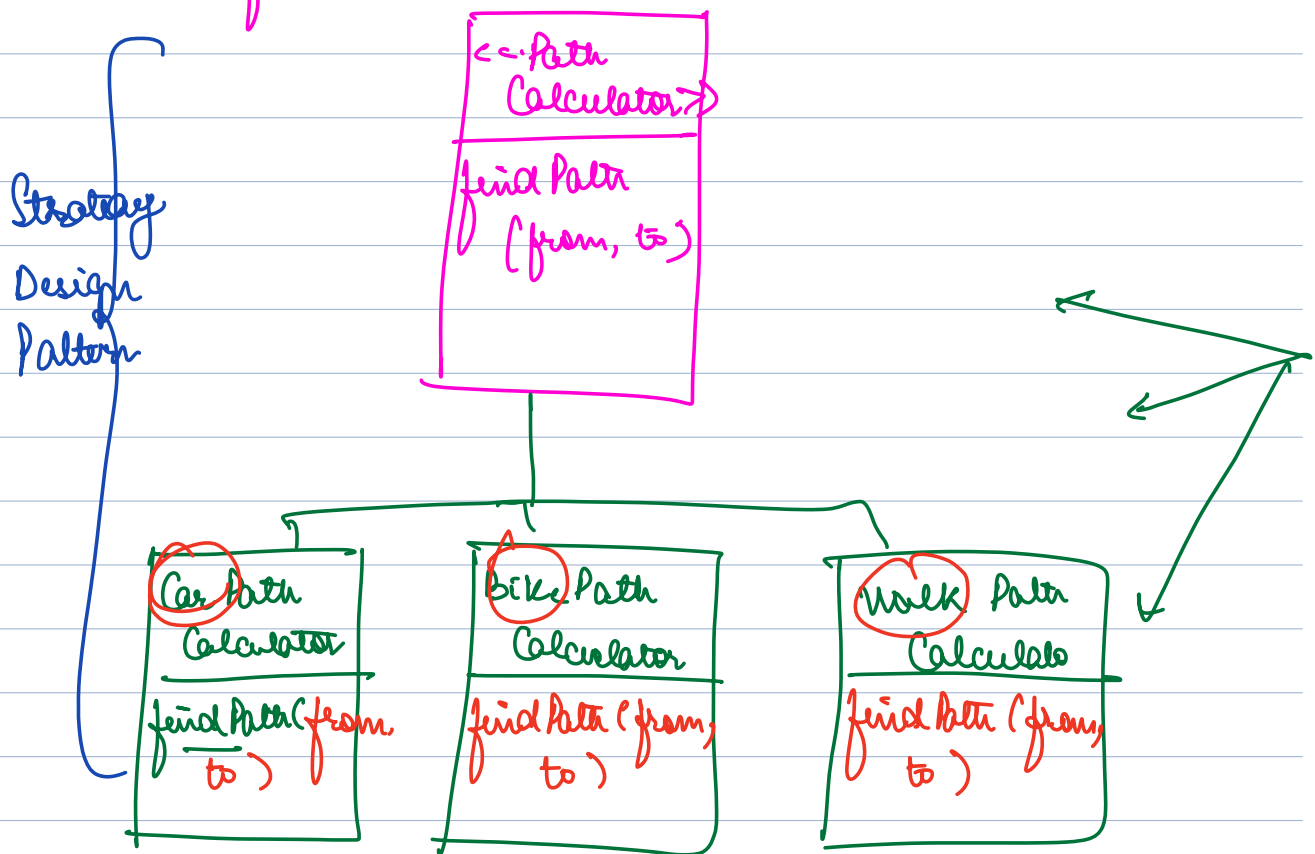
Violates SRP and OCP

=> Whenever there are multiple ways to perform a behaviour, instead of coding a behaviour in an entity, create a sp. class for each type of that behaviour.





⇒ Make each class implement a common interface



Path Calculator Factory {

PathCalculator createPathCalcForMode (Mode mode) {
if (mode == CAR)
return new CarPathCalculator();

}

class Google Maps {

findPath (from, to, mode) {

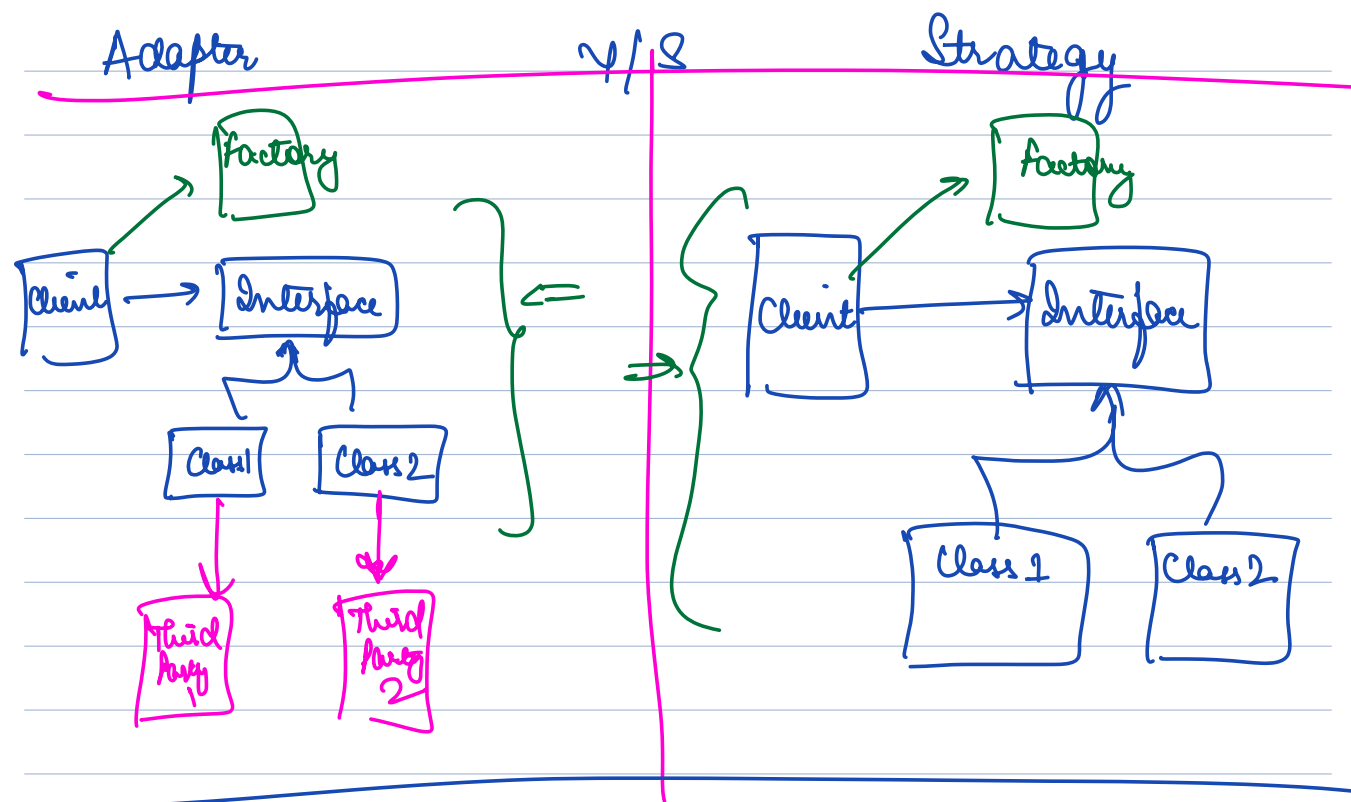
⇒

PathCalculator pc = PCF.create — (mode)

pc.findPath (from, to)

}

}



Factory: Creating obj of correct subclass

Adapter: Talking to a 3rd party class

Strategy: Implementing beh with multiple variants

Assg^m

⇒ Rainwater Trapping Problem

→ coinbase

→ stripe

	TC	SC	
⇒	$O(N^2)$	$O(1)$	
	$O(N)$	$2 \times O(N)$	
	$O(N)$	$O(N)$	
	$O(N)$	$O(1)$	

(Mix DSA problems with code quality)
 ← { diff way to calc } problems
 (Amount of water trapped)

class Solution {

int solve (heights, algo) {

Algo {

N2O1

NON

NO2N

ND1

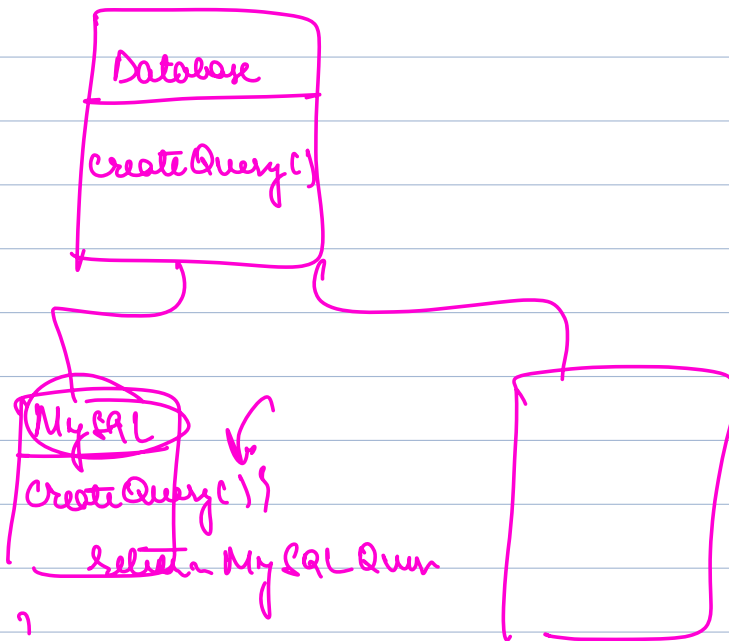
← RainWater Trapping
Solⁿ →

DN2O1 Solⁿ

1. List < Step > find data (from, to, mode) {

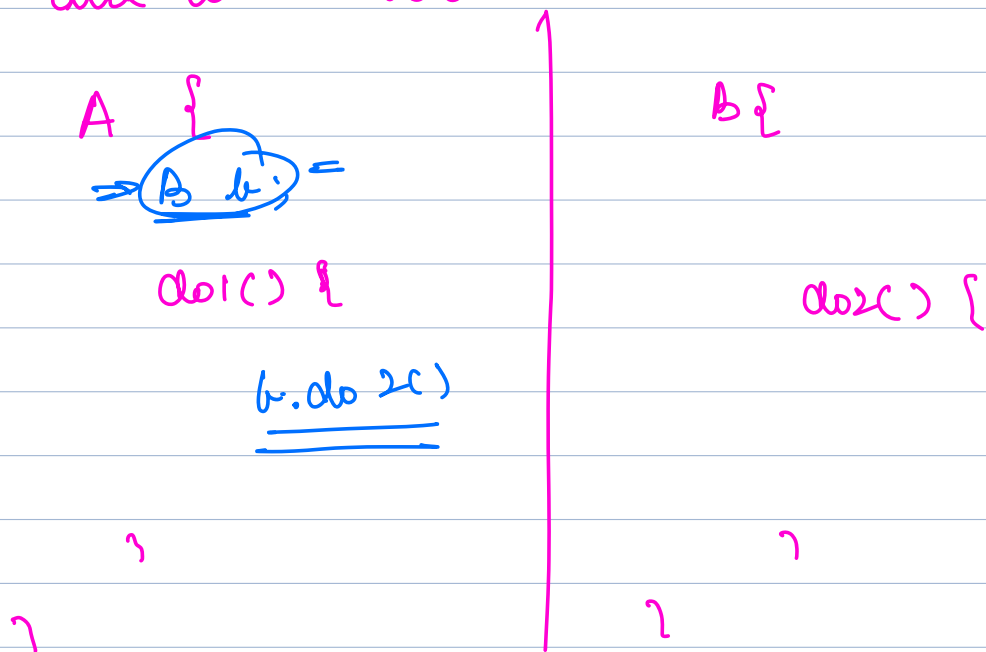
—
←
—
←
—

}



Observer Design Pattern

- reverse the way we think about implementing functionalities
- if a class had to perform a funcⁿ that was there in another class, how did it do that



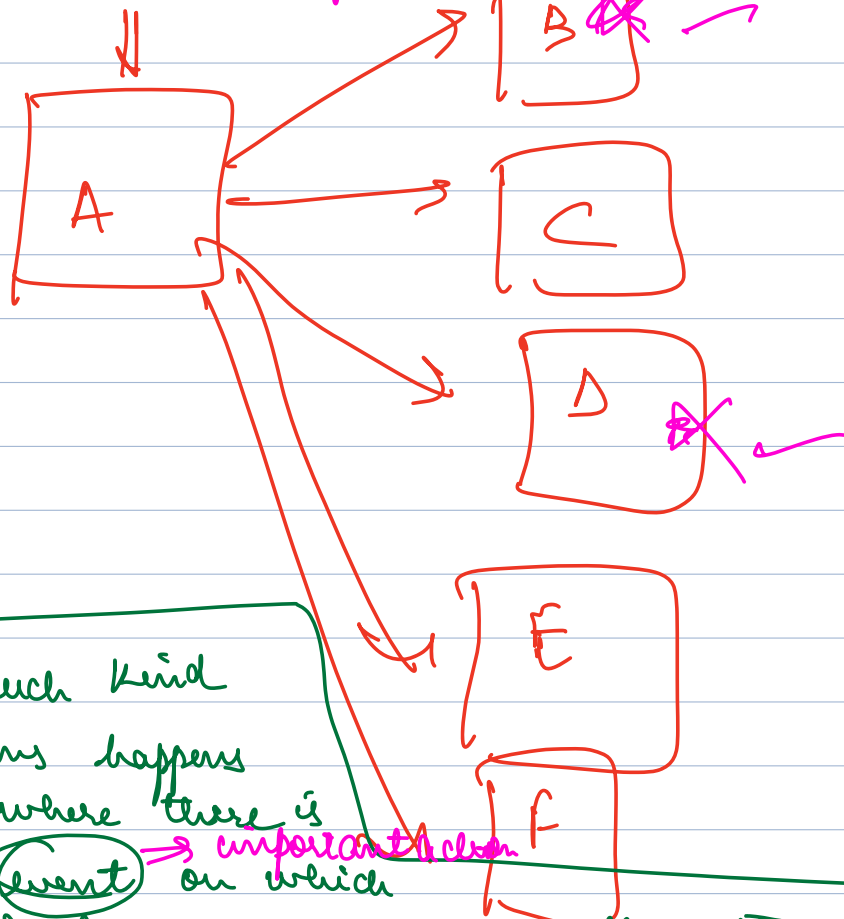
Way 1: Have an attr of that class and call methods on it.

BUT BUT BUT

what if I have to call many functionalities across diff classes.

- ① too many attrs in class?
- ② need to make changes in code / redeploy whenever I may want to

change the classes that are called



typically such kind of a problems happens in systems where there is a special event on which something is happening & have to update multiple things

→ FLIPKART

Flipkart {
Order Cancelled () {

Order Placed () {
WarehouseManager.updateInv()

invoice gen. genInvoice()
→ { emailService.sendEmail()
smsService.
course.notify()

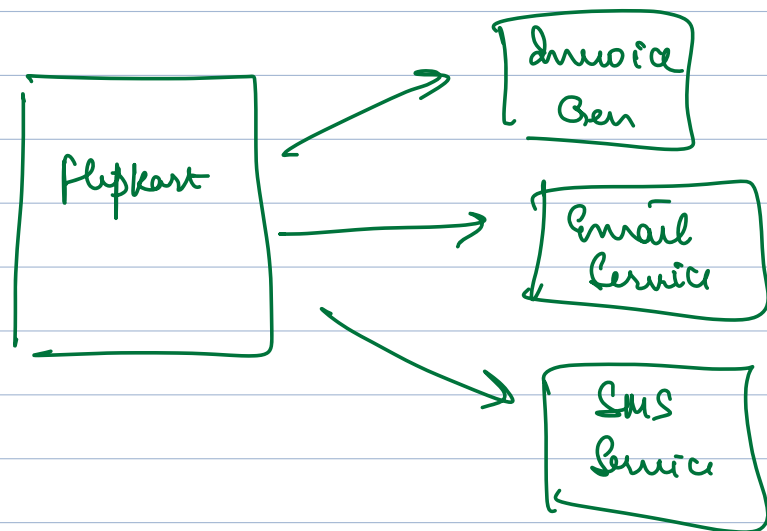
→ SCALER

private MentorService service

userPaid () {

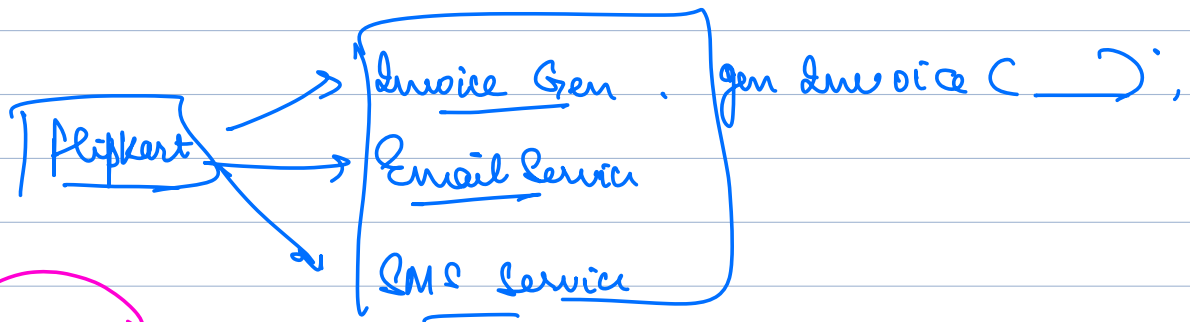
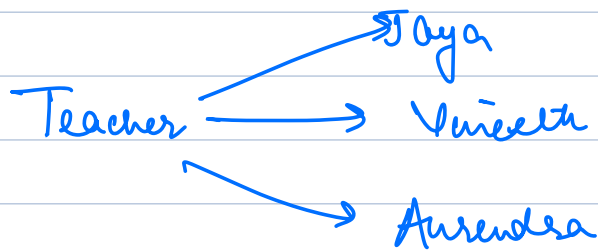
→ mentorService.notifyMentor(userId)
→ emailService.sendEmail()
→ smsSender.send()
notifyService

As soon as an event happens, I call many other services to do what they should do on that event happening.

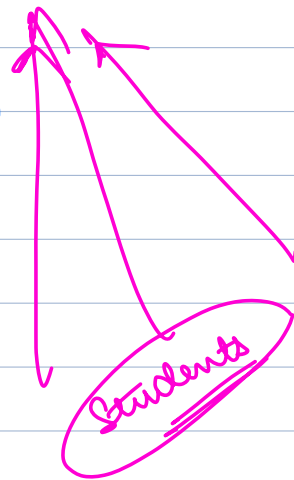
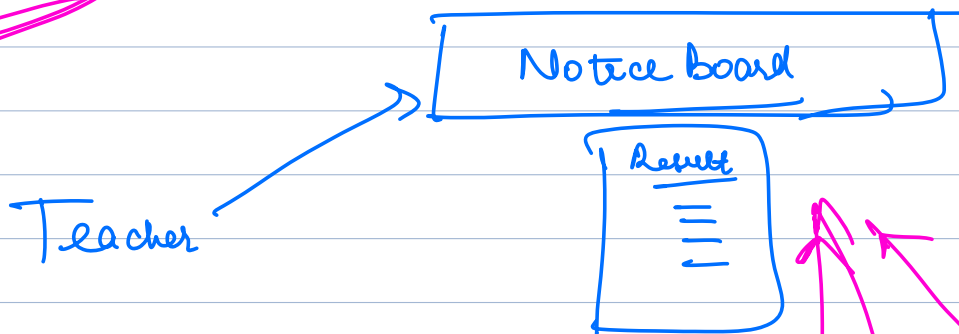


⇒ Observer design pattern solves the problem solve of a class calling many other classes on occurrence of a special behaviour that needs updating multiple systems.

Give Exam → Teacher Evaluated → Show Result.



Observer



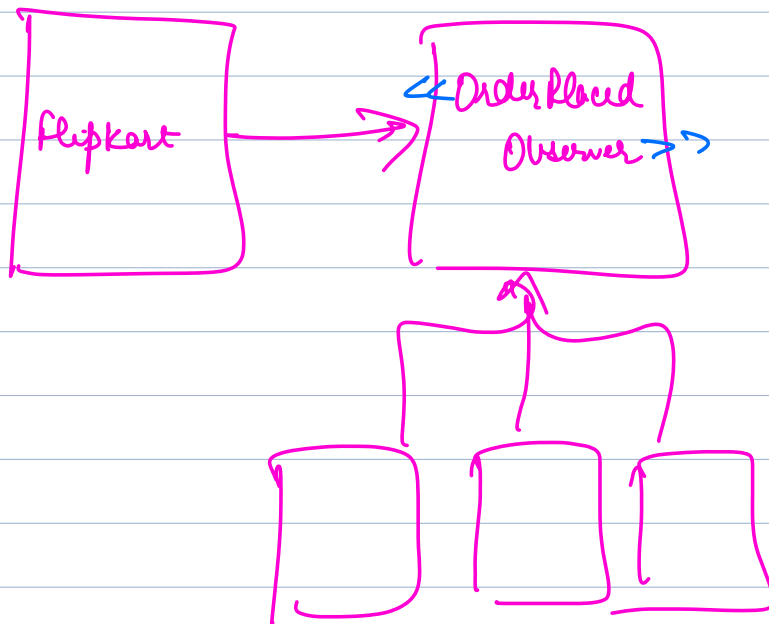
(Observers of notice board)

⇒ As soon as something new gets put on the notice board every student can know about it.

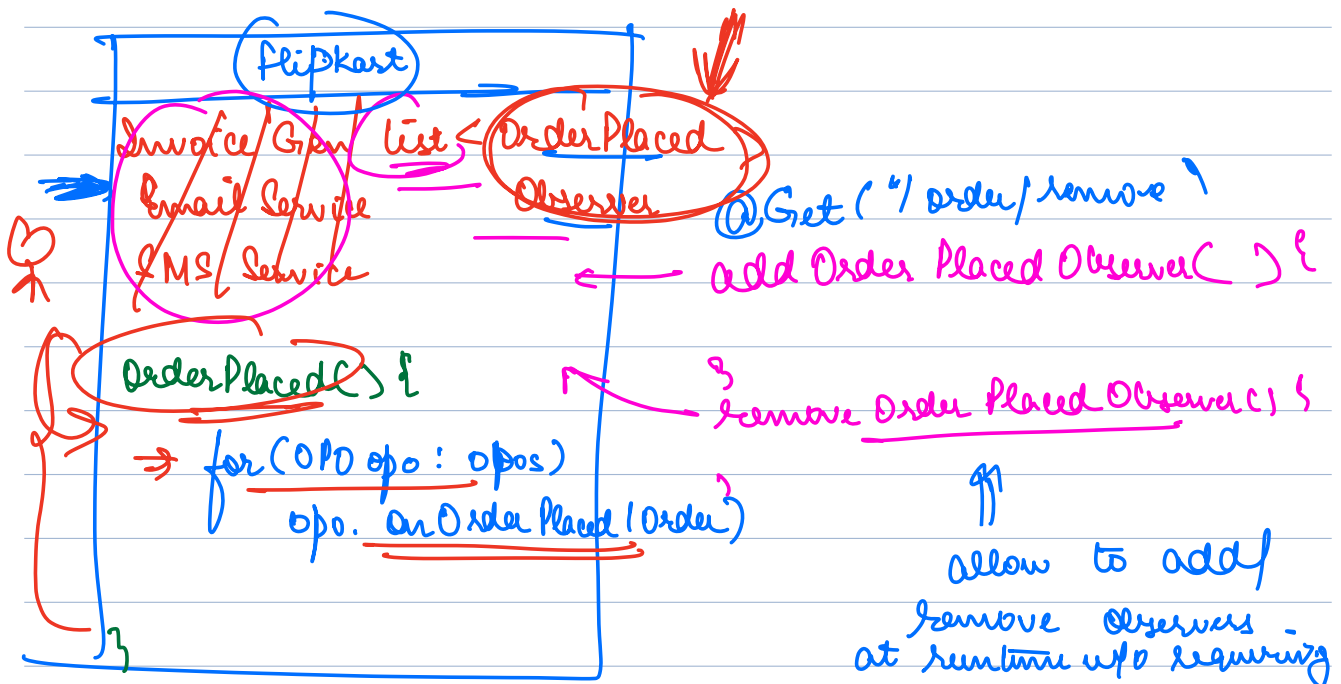
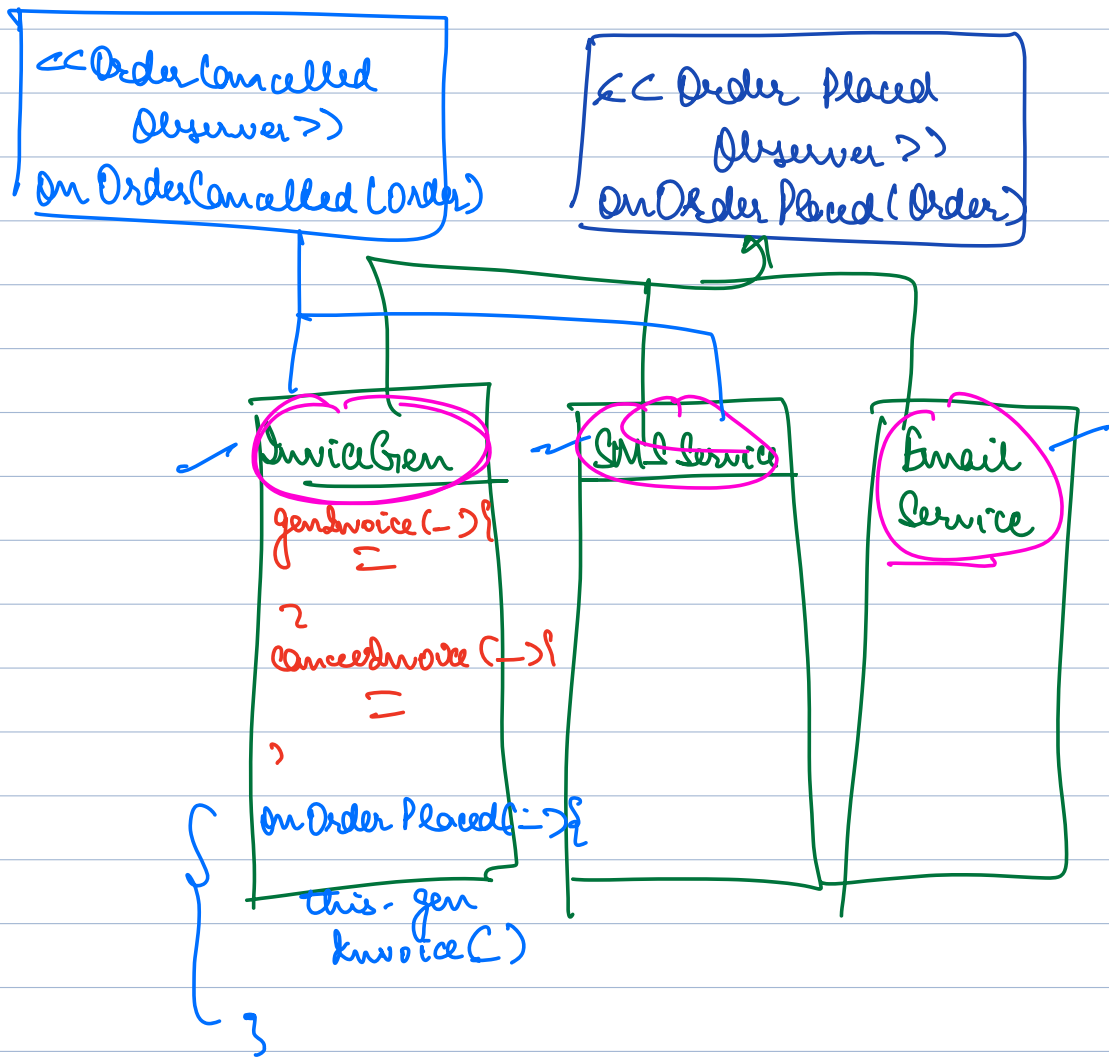
Observer

Whenever on occurrence of a special event we have to call many ~~ids~~ external dependencies

- ① I create an interface that every dependency should implement



Dependency
inversion

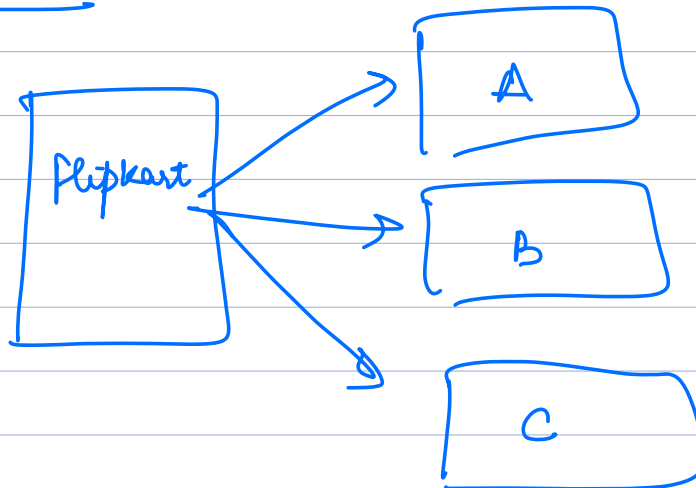


Any code change

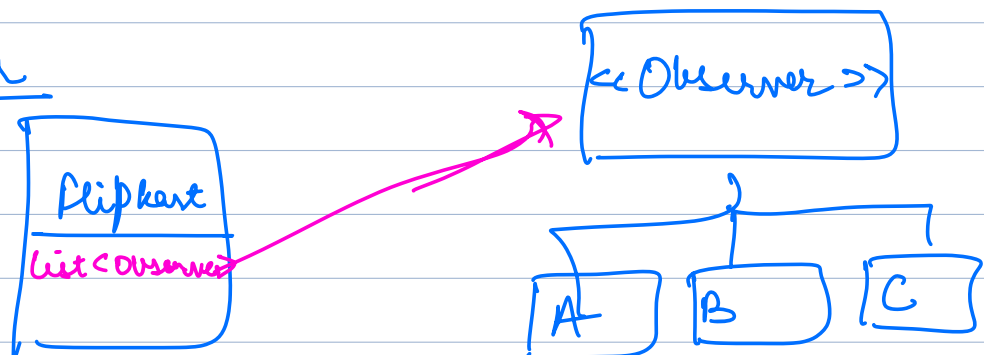
```
class Flipkart {  
    list < OrderPlaced Observer > _____  
    list < OrderCancelled Observer > _____  
  
    Add Order Placed — ( ) { }  
    remove Order Placed — ( ) { }  
}
```

}

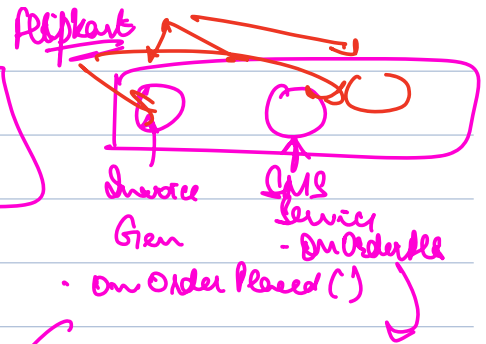
w/o observer



with Observer



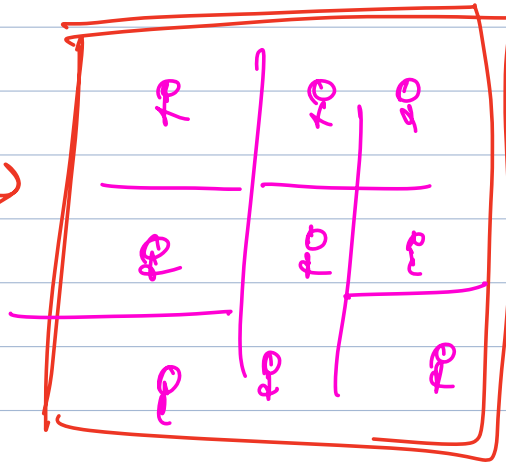
Break till 10:34 PM



FACADE DESIGN PATTERN

Boundary / External of something

Facade



⇒ Clean outside so as to hide complex internals

⇒ if code starts becoming too complex
(if you have to talk to a lot
of dependencies and work with
them)

⇒ don't make your code too complex ⇒ abstract
out things

```
class Flipkart {
```

→ Move that complexity into sep classes.

```
    orderPlaced() {
```

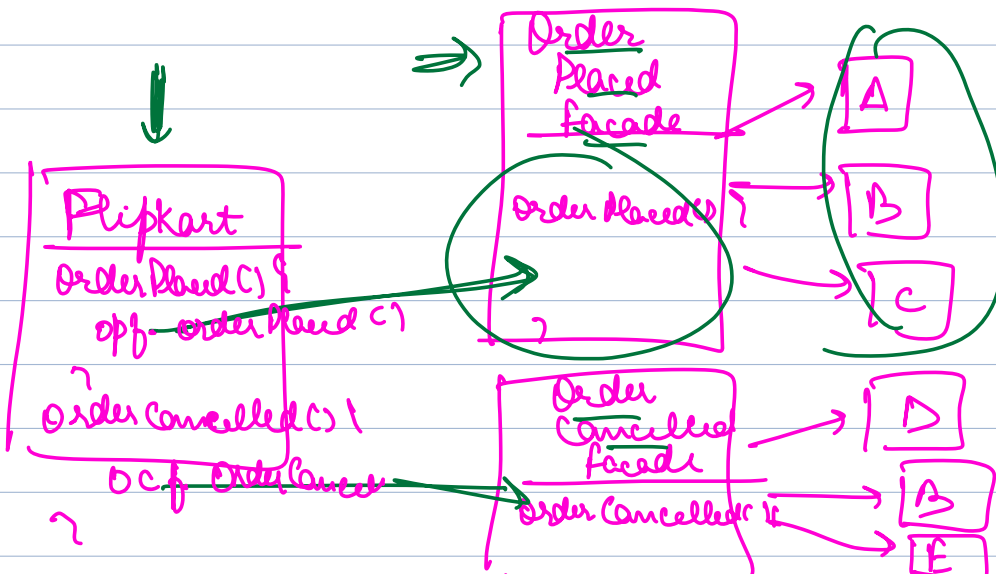
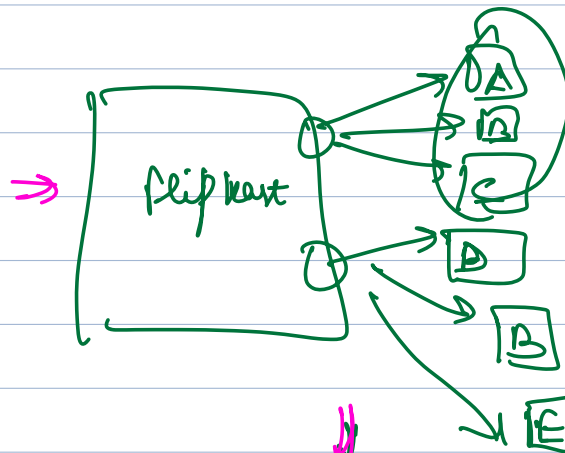
```
        a. do1()
```

```
        b. do2()
```

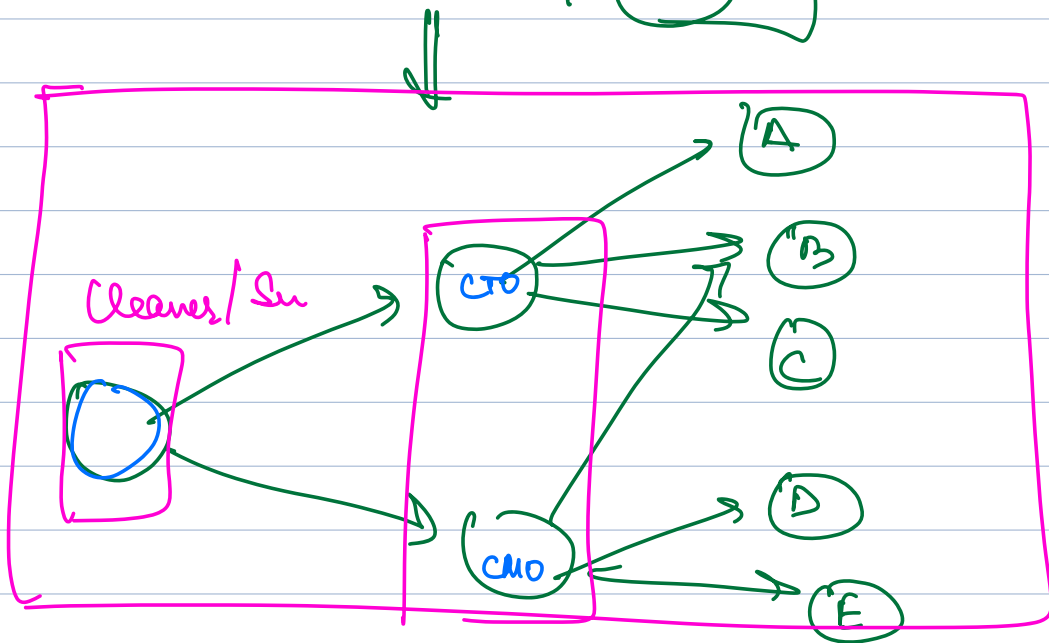
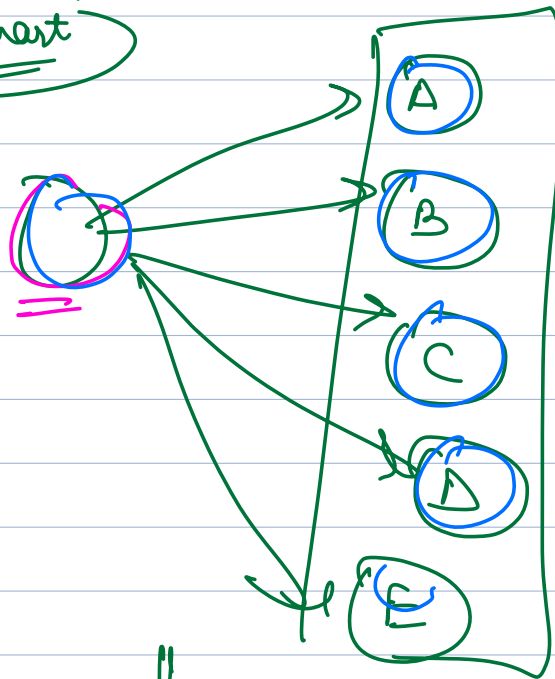
```
        c. do3()
```

```
    } ( ——— )
```

```
    return a + b + c
```



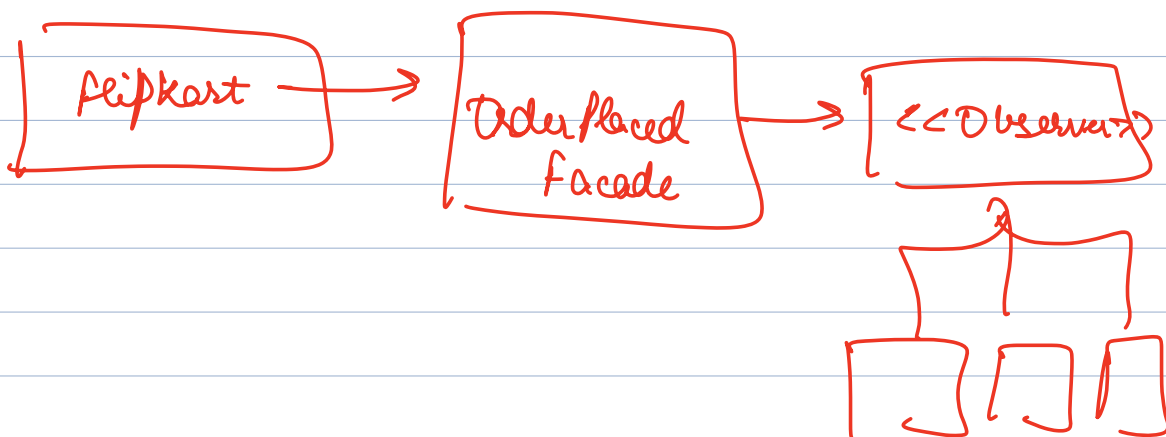
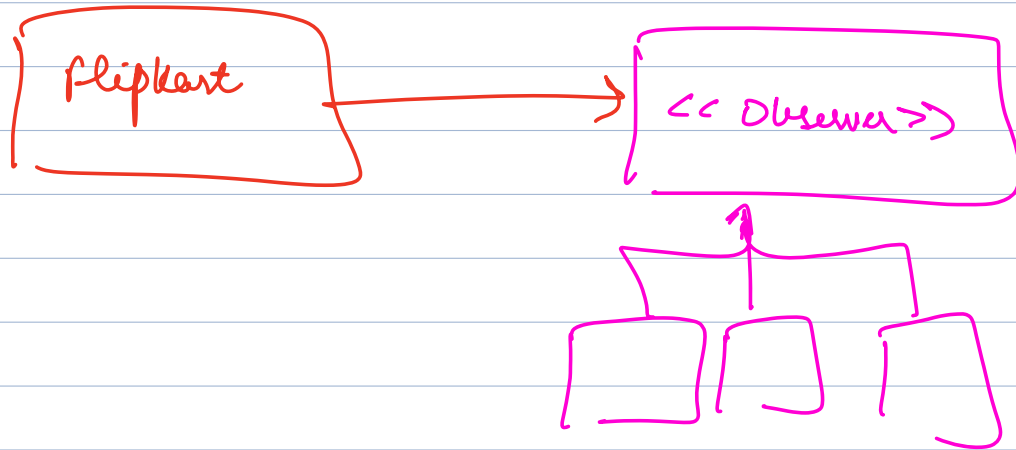
Dep Chart



```
class Flipkart {  
    OrderPlaced Facade  
    OrderCancelled Facade  
}
```

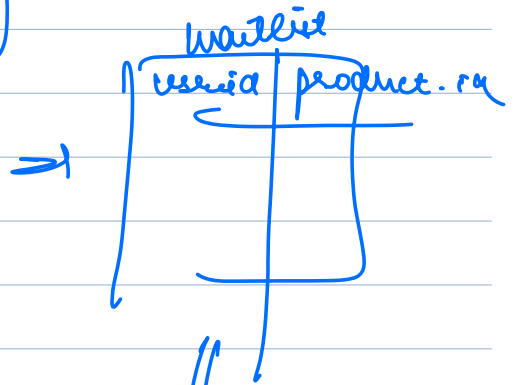
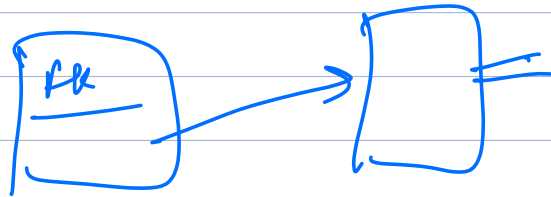
Facade (Structural Design Pattern)

⇒ If one class becomes too complex, break it into multiple classes



Facade : Refactoring code .

: break a class into multiple classes



Flipkart {

addProduct() {

list < Users >

(DB.getUsersWaitingForProduct)

for (User u: users)

notifyService.notify(u)

}

}